

Q: Is the hook a “custom accounting hook”? Does it use beforeSwapReturnDelta or afterSwapReturnDelta?

In Fiet, there are **two Uniswap v4 hooks per market**, because each market is expressed as:

- a **proxy (“native”) pool** on the underlying pair (e.g. ARB/USDT), and
- a **core (“synthetic”) pool** on the LCC pair (e.g. lccARB/lccUSDT).

Those two hooks serve **different purposes**, and they use **different v4 hook permissions**.

Proxy pool hook (contracts/evm/src/ProxyHook.sol): swap-proxy / custom delta

- **Yes, this hook uses beforeSwapReturnDelta.**
 - ProxyHook.getHookPermissions() sets beforeSwapReturnDelta: true and afterSwapReturnDelta: false.
- **What it’s doing:** it *overrides the proxy pool’s swap accounting* by returning a custom BeforeSwapDelta.
 - Concretely, ProxyHook._beforeSwap(...) executes the swap against the **core LCC pool** (poolManager.swap(corePoolKey, ...)) and then constructs a BeforeSwapDelta so the user’s proxy-pool swap is “fulfilled” by the core pool outcome.
 - During that process it mints/burns and settles LCCs via LiquidityHub (wrap/unwrap), so the underlying-facing proxy pool can route depth to/from the LCC core pool.

Interpretation: this is a **swap routing / delta override hook**, not the primary place where Fiet’s VTS fee-sharing and position accounting is computed.

Core pool hook (contracts/evm/src/CoreHook.sol): VTS accounting + fee-sharing materialisation

- **No, this hook does *not* use beforeSwapReturnDelta or afterSwapReturnDelta.**
 - CoreHook.getHookPermissions() sets beforeSwapReturnDelta: false and afterSwapReturnDelta: false.
- **Where the “custom accounting” happens instead:**
 - **Swap-driven accounting** is handled via standard beforeSwap/afterSwap:
 - * CoreHook._afterSwap(...) forwards swap deltas and pre-swap state into vtsOrchestrator.afterCoreSwap(...), which is where tick-indexed growth accounting is applied to attribute inflows/deficits to in-range liquidity.
 - **Fee-sharing / adjustment materialisation** happens via **liquidity-modification return deltas**:
 - * CoreHook enables afterAddLiquidityReturnDelta: true and afterRemoveLiquidityReturnDelta: true.

- * In `_afterAddLiquidity(...)` and `_afterRemoveLiquidity(...)`, the hook calls `vtsOrchestrator.processPosition(...)` and returns a `feeAdj BalanceDelta`. That returned delta is the mechanism used to **slash or bonus** LPs at modify-liquidity time (as described in the tick-indexed coverage + fee-sharing spec), i.e. “the hook takes” (positive delta) or “the hook gives” (negative delta).
- It also enforces the “settle growths before liquidity mutations” invariant by calling `vtsOrchestrator.settlePositionGrowths(...)` in `beforeAddLiquidity` and `beforeRemoveLiquidity`, preventing retroactive capture of previously-accrued growth by newly-added liquidity.

Interpretation: the core hook is best described as Fiet’s **accounting and fee-adjustment hook**, but it achieves that through `beforeSwap/afterSwap` plus `afterAddLiquidityReturnDelta/afterRemoveLiquidityReturnDelta` — **not** through `beforeSwapReturnDelta/afterSwapReturnDelta`.

Bottom line

- **ProxyHook (native/proxy pool):** uses `beforeSwapReturnDelta` to proxy swaps into the core LCC pool and return a custom delta.
- **CoreHook (synthetic/core LCC pool):** is the **accounting hook**, but uses `beforeSwap/afterSwap` for swap-linked accounting and `afterAddLiquidityReturnDelta/afterRemoveLiquidityReturnDelta` to materialise fee-sharing adjustments; it does **not** use `beforeSwapReturnDelta` nor `afterSwapReturnDelta`.